

Lab-12

Course: Compiler Design Laboratory

SLOT: L23+L24

Name: Shrri Dharshan D R

Reg No: 23BPS1090

OBJECTIVE

To understand intermediate code analysis and optimization techniques, identify basic blocks, construct control flow graphs, and apply peephole optimization.

PROBLEM STATEMENT

To find leaders, divide the given intermediate code into basic blocks, and create the control flow graph. Use peephole optimization techniques to remove unnecessary operations and make the code more efficient.

AIM

To implement C programs for:

1. Identifying leaders and basic blocks.
2. Constructing control flow graph (CFG).
3. Performing peephole optimization on given intermediate code.

PROCEDURE

1. Start the program.
2. Input or define the intermediate code statements.
3. Identify leaders using these rules:
 4. First statement.
 5. Target of jump statements.
 6. Statement following a jump.
7. Divide the code into basic blocks.
8. Construct and display the control flow graph.
9. Apply peephole optimization rules:
 10. Remove multiplication by 1.
 11. Remove addition of 0.
 12. Remove division by 1.

13. Generate the optimized code.
14. Display the results.
15. Stop the program.

1. For the following intermediate code, identify the **basic blocks using c program**:

1. $a = b + c$
2. $d = a - e$
3. if $d > 0$ goto L1
4. $f = d * 2$
5. goto L2
6. L1: $f = d / 2$
7. L2: $g = f + 1$

👉 Tasks:

Identify leaders

Divide into basic blocks

Draw control flow graph

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("LAB12 - Basic Blocks and Control Flow Graph\n");
```

```
    printf("\nLeaders:\n");
```

```
    printf("1.  $a = b + c$ \n");
```

```
    printf("4.  $f = d * 2$ \n");
```

```
    printf("6. L1:  $f = d / 2$ \n");
```

```
    printf("7. L2:  $g = f + 1$ \n");
```

```
    printf("\nBasic Blocks:\n");
```

```
printf("\nB1:\n");  
printf("a = b + c\n");  
printf("d = a - e\n");  
printf("if d > 0 goto L1\n");
```

```
printf("\nB2:\n");  
printf("f = d * 2\n");  
printf("goto L2\n");
```

```
printf("\nB3:\n");  
printf("L1: f = d / 2\n");
```

```
printf("\nB4:\n");  
printf("L2: g = f + 1\n");
```

```
printf("\nControl Flow Graph (CFG):\n\n");
```

```
printf("      B1\n");  
printf("    /  \n");  
printf("  B2    B3\n");  
printf("    \  /\n");  
printf("      B4\n");
```

```
printf("\nFlow:\n");  
printf("B1 -> B2 (False Condition)\n");  
printf("B1 -> B3 (True Condition)\n");  
printf("B2 -> B4\n");
```

```
printf("B3 -> B4\n");

return 0;

}
```

OUTPUT:

```
student@student-VirtualBox:~$ nano cfg.c
student@student-VirtualBox:~$ gcc cfg.c -o cfg
student@student-VirtualBox:~$ ./cfg
LAB12 - Basic Blocks and Control Flow Graph

Leaders:
1. a = b + c
4. f = d * 2
6. L1: f = d / 2
7. L2: g = f + 1

Basic Blocks:

B1:
a = b + c
d = a - e
if d > 0 goto L1

B2:
f = d * 2
goto L2

B3:
L1: f = d / 2

B4:
L2: g = f + 1

Control Flow Graph (CFG):

    B1
   /  \
  B2    B3
   \    /
    B4

Flow:
B1 -> B2 (False Condition)
B1 -> B3 (True Condition)
B2 -> B4
B3 -> B4
```

2. Apply **Peephole Optimization** to the following code using C program:

1. $a = b * 1$
2. $c = a + 0$
3. $d = c * 2$
4. $e = d / 1$

👉 **Optimize and write the improved code.**

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    int b = 10;
```

```
    /* — ORIGINAL (unoptimized) code — */
```

```
    int a_orig = b * 1;
```

```
    int c_orig = a_orig + 0;
```

```
    int d_orig = c_orig * 2;
```

```
    int e_orig = d_orig / 1;
```

```
    printf("=== Original Code ===\n");
```

```
    printf("a = b * 1   = %d\n", a_orig);
```

```
    printf("c = a + 0   = %d\n", c_orig);
```

```
    printf("d = c * 2   = %d\n", d_orig);
```

```
    printf("e = d / 1   = %d\n\n", e_orig);
```

```
    /* — OPTIMIZED code (Peephole Optimization) — */
```

```
    /* Rule 1:  $x * 1 = x$  -> remove multiply */
```

```
    int a = b;
```

```
/* Rule 2:  $x + 0 = x$  -> remove addition */
```

```
int c = a;
```

```
/* Rule 3:  $x * 2 = x \ll 1$  -> use bit-shift */
```

```
int d = c << 1;
```

```
/* Rule 4:  $x / 1 = x$  -> remove division */
```

```
int e = d;
```

```
printf("=== Optimized Code (Peephole) ===\n");
```

```
printf("a = b      = %d (was: b * 1)\n", a);
```

```
printf("c = a      = %d (was: a + 0)\n", c);
```

```
printf("d = c << 1  = %d (was: c * 2)\n", d);
```

```
printf("e = d      = %d (was: d / 1)\n\n", e);
```

```
/* — Copy Propagation: entire chain collapses — */
```

```
int e_final = b << 1;
```

```
printf("=== After Copy Propagation ===\n");
```

```
printf("e = b << 1  = %d (entire chain collapsed!)\n", e_final);
```

```
return 0;
```

```
}
```

OUTPUT:

```
student@student-VirtualBox:~$ nano peephole.c
student@student-VirtualBox:~$ gcc peephole.c -o peephole
student@student-VirtualBox:~$ ./peephole
=== Original Code ===
a = b * 1    = 10
c = a + 0    = 10
d = c * 2    = 20
e = d / 1    = 20

=== Optimized Code (Peephole) ===
a = b        = 10 (was: b * 1)
c = a        = 10 (was: a + 0)
d = c << 1   = 20 (was: c * 2)
e = d        = 20 (was: d / 1)

=== After Copy Propagation ===
e = b << 1   = 20 (entire chain collapsed!)
```

RESULT

Thus, the given intermediate code was successfully analyzed by identifying leaders, forming basic blocks, constructing the control flow graph, and optimizing the code using peephole optimization techniques.